Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Kayan nokta gösterimi (floating point notation) tekrarı
  - 8 bitlik örnekte bit grubu üç alana ayrılır: işaret biti (sign bit, 1 bit), üs alanı (exponent field, 3 bit) ve mantis alanı (mantissa field, 4 bit).
  - İşaret biti sayının pozitif ya da negatif olduğunu gösterir. Üs alanı noktanın kaç basamak ve hangi yöne kayacağını gösterir (excess-4 gösterimiyle). Mantis alanı sayının tam ve kesirli kısmının birlikte bit karşılığıdır.
  - Bir kesirli sayıyı bu biçime çevirmenin adımları:
    1. İşaret bitini yaz: sayı pozitifse 0, negatifse 1.
    2. Sayıyı ikilik kesir olarak normal (noktalı) biçimde yaz.
    3. Mantise, ilk 1 bitinden başlayarak yerleştir. Baştaki 0'lar atılır, mantisin sağında boş kalan bitler 0 ile doldurulur.
    4. Noktanın mantis başından gerçek konumuna kaç basamak kaydığını bul ve bu değeri excess-4 ile üs alanına yaz.
  - Sağlama: işaret biti okunur, mantisin başına nokta konur, üs değeri kadar nokta kaydırılır ve çıkan değer hesaplanır.
  - Nokta sola kaydırıldığında boşalan basamağa 0 gelir. Bitler kaymaz, yalnızca nokta kayar.
- Tahtadaki örnekler (ekrandan, 00:36:00)
  - Örnek 1: 2 3/4
    - İşaret: 0. İkilik yazım: 10.11.
    - Mantis: 1011. Nokta mantis başından 2 basamak sağa kayıyor → üs +2 → excess-4'te 6 = 110.
    - Sonuç: 0 | 110 | 1011. Sağlama: .1011 → 10.11 = 2 + 1/2 + 1/4 = 2 3/4.
  - Örnek 2: 3/8
    - İşaret: 0. İkilik yazım: .011 (1/4 + 1/8 = 3/8).
    - Mantis ilk 1'den başlar: 1100 (sağ taraf 0 ile doldurulur).
    - Nokta mantis başından 1 basamak sola kayıyor → üs −1 → excess-4'te 3 = 011.
    - Sonuç: 0 | 011 | 1100. Sağlama: .1100 → nokta bir sola kayar → .01100 = 3/8.
  - Örnek 3 (truncation error): 2 5/8
    - İkilik yazım: 10.101 → 5 anlamlı bit gerekir.
    - Mantis yalnızca 4 bit olduğundan son bit düşer: mantis 1010, üs +2 (110).
    - Sonuç: 0 | 110 | 1010. Sağlama: .1010 → 10.10 = 2 4/8 = 2 1/2.
    - 2 5/8 yazılmak istenmiş ama 2 4/8 saklanmıştır. 1/8'lik bit kaybolmuştur.
- Kesme hatası (truncation error / round-off error)
  - Mantis alanı yetersiz olduğunda sayının bazı bitleri düşer ve sayı daha düşük hassasiyetle saklanır.
  - Bilgisayar bu durumda hata vermez. İşlem kaybedilen hassasiyetle devam eder. Bu yüzden iki farklı sistemin sonuçları küçük farklarla ayrışabilir.
  - 1/3 veya 1/20 gibi sayılar 2'nin kuvvetleri cinsinden sınırlı bitle tam ifade edilemez. Daha hassas kesirler için daha geniş mantis ve üs alanı gerekir.
- Tek ve çift duyarlıklı kayan nokta (ekrandan, 00:41:50)
  - Single precision floating point: 32 bit. 1 bit işaret, 8 bit üs, 23 bit mantis. Yaklaşık 10^−37 ile 10^38 aralığını ve noktadan sonra yaklaşık 7 ondalık basamak (decimal digit) hassasiyeti sağlar.
  - Double precision floating point: 64 bit. Noktadan sonra yaklaşık 15 ondalık basamak

hassasiyeti sağlar.
  - Bit sayısı arttıkça üs ve mantis alanları büyür. İşaret biti her zaman 1 bittir.
  - C dilinde `float` ve `double` için işaretsiz (`unsigned`) tanım yapılamaz. Kesirli sayılar her zaman işaretlidir.
- Farklı sistemler, farklı kurallar
  - Her bilgi (komut, resim, harf, sayı) 0 ve 1 kombinasyonlarıyla saklanır, ancak bunun için kurallar vardır. Her işlemci mimarisi aynı gösterimi (örneğin ikiye göre tümleyen) kullanmayabilir.
- Veri ve bilgi
  - Veri (data) anlamlandırıldığında bilgi (information) olur.
  - Her veri bilgi değildir, ama her bilgi veri içerir.
- Bilgisayarda saklanan veri türleri
  - Sayılar (pozitif, negatif, kesirli), karakterler (harfler, rakamlar, noktalama işaretleri, boşluk, enter gibi), renkler, CPU komutları (instructions), sesler, resimler (image), videolar, dosyalar, web sayfaları ve programlar.
- ASCII tablosu (ekrandan, 01:13:35)
  - Karakterleri saklamak için her karaktere bir sayı karşılığı veren tablolar kullanılır. En bilineni ASCII'dir (American Standard Code for Information Interchange). Hoca bu tabloyu 8 bitlik ve 256 sembollük olarak anlatır.
  - Tabloda rakamlar, büyük harfler, küçük harfler, noktalama işaretleri ve semboller bulunur.
  - Referans değerler: '0' = 48, dolayısıyla '9' = 48 + 9 = 57. 'A' = 65, 'B' = 66.
  - Karakterler tek tırnak içinde yazılır: `'3'`, `'a'`, `' '`.
  - Sayı olarak 3 ile karakter olarak '3' farklıdır:
    * 3 (sayı) → 00000011
    * '3' (karakter) → 51 → 00110011
  - "123" üç karakterdir (3 byte). 123 ise tek bir sayıdır.
- UTF-8
  - ASCII'deki 256 sembol, Türkçedeki ğ, ş, ü gibi karakterleri, Kiril alfabesini veya Çince gibi dilleri karşılamaz.
  - UTF-8 (Unicode Transformation Format) standart sembolleri ASCII ile aynı kodlarla ifade eder. Ek semboller için kullanılan alana göre 24 ya da 32 bit kullanarak çok daha fazla sembolü kodlayabilir. Bu sayede tüm dillerdeki karakterler ifade edilebilir.
- Sayıyı sayı olarak ve karakter olarak saklamak
  - 333 karakter olarak saklanırsa her rakam 1 byte olduğundan 3 byte kaplar. Sayı olarak saklanırsa 1 byte'a sığmaz (en fazla 255), bu yüzden 2 byte kaplar.
  - Bellekte 9, 10, 12 bit gibi alanlar kullanılmaz. Alanlar 8'in katlarıdır (8, 16, 24, 32 bit).
  - 4782 karakter olarak 4 byte, sayı olarak 2 byte kaplar. 2 byte ile işaretsiz 65.536 farklı değer, işaretli olarak yaklaşık 32.768'e kadar sayı ifade edilebilir.
  - Metin içinde geçen sayılar karakter olarak saklanır. Program içinde sayısal işlem yapılacak değerler daha az yer kapladığı için sayı tipindeki değişkenlerde tutulmalıdır.
- Resim, video ve sesin saklanması
  - Resmin en küçük birimi pikseldir (picture element). Resim, yüksekliği (height) ve genişliği (width) olan bir matris olarak düşünülür.
  - 1920×1080'lik bir resimde her piksel 1 byte ile ifade edilirse resim 1920 × 1080 byte yer kaplar. Saniyede 30 kare (30 fps) olan bir videoda bu değer saniye başına 30 ile çarpılır.
  - Ses genellikle tek boyutlu bir dizi (array) olarak saklanır. Değerler sesin şiddetini gösterir. Frekans ve genlik (amplitude) gibi kavramlar ayrıntıdadır.
  - Aynı tipteki veriler bellekte kesintisiz olarak dizilerle saklanır. Tek bir değer ise tek bir değişken alanında tutulur.
- Veri sıkıştırma
  - Sıkıştırma, verinin daha az yer kaplayarak saklanmasıdır. Dosya sistemleri az erişilen dosyaları arka planda sıkıştırarak saklayabilir.
  - Kayıpsız (lossless) sıkıştırma: veri açıldığında birebir aynısı elde edilir. Örneğin 100 byte 10 byte'a düşürülür ve tekrar aynı 100 byte'a açılır.
  - Kayıplı (lossy) sıkıştırma: belirli oranda veri kaybedilir. Örnek: MP3, insanın duyamadığı sesleri saklamaz. Görüntüde de gözün ayırt edemediği renk farkları atılabilir. Kayıplı sıkıştırma çoğunlukla ses, görüntü ve videoda kullanılır.

Hocanın Özellikle Vurguladığı Kısımlar

- Kayan nokta dönüşümünü adım adım yapın
  - Sıra her zaman şöyledir: önce işaret biti, sonra sayının normal ikilik yazımı, sonra mantis, en son üs. Akıldan ve adım atlayarak çözmek hataya yol açar.
- Mantis mutlaka ilk 1'den başlar
  - Baştaki 0'lar mantise yazılmaz. Bu kural uygulanmazsa sayı yanlış saklanır.
- Sağlama yapın
  - Elde edilen bit grubundan geri dönülerek aynı sayı elde ediliyorsa dönüşüm doğrudur.
- Truncation error bilgisayara hata verdirmez
  - Hassasiyet kaybı fark edilmeden işlemlere yansır. Sonuçlar arasında küçük farklılıklar bu yüzden oluşabilir.
- ASCII değerlerini ezberlemeyin
  - Referans noktalarını kullanın: '1' ile '0' arasındaki fark her zaman 1'dir, '9' ile '0' arasındaki fark 9'dur, 'Z' ile 'A' arasındaki fark alfabedeki harf sayısıyla ilişkilidir. Problemler karakter sabitleri kullanılarak çözülür.
- Derslerden sonra çözümleri tekrar edin
  - Hoca, sorulara verilen cevaplardan derslerin arkasından çözümlere yeterince vakit ayrılmadığını gözlemler. Her ders sonrası örnekleri kendi başınıza tekrar çözmeniz gerekir.

Kısa Tekrar Notları

- 8 bit kayan nokta: işaret (1) | üs (3, excess-4) | mantis (4).
- Adımlar: işaret → ikilik yazım → mantis (ilk 1'den) → üs (noktanın kayma miktarı).
- 2 3/4 → 0|110|1011. 3/8 → 0|011|1100. 2 5/8 → 0|110|1010 (= 2 4/8, truncation error).
- Truncation (round-off) error: mantis yetmeyince bit düşer, hata mesajı oluşmaz.
- Single precision: 32 bit = 1 + 8 + 23, ~7 ondalık basamak. Double precision: 64 bit, ~15 ondalık basamak.
- Veri anlamlandırılınca bilgi olur.
- ASCII: '0' = 48, 'A' = 65. Karakterler tek tırnakta yazılır. 3 ≠ '3' (00000011 ≠ 00110011).
- UTF-8: ASCII'yi kapsar, tüm dillerin karakterlerini kodlayabilir.
- 333 → karakter 3 byte, sayı 2 byte. 4782 → karakter 4 byte, sayı 2 byte.
- Resim = piksel matrisi. Ses = tek boyutlu dizi.
- Lossless = birebir geri açılır. Lossy = kayıplıdır (MP3).

Detaylı Açıklamalar

Ders, kayan nokta gösteriminin örneklerle pekiştirilmesiyle başlar. 8 bitlik örnek biçimde ilk bit işaret biti, sonraki üç bit üs alanı, son dört bit mantis alanıdır. Bir kesirli sayı bu biçime çevrilirken önce sayının işaretine bakılır. Sonra sayı ikilik sistemde noktalı biçimde yazılır. Bu yazımdaki anlamlı bitler, ilk 1'den başlanarak mantise yerleştirilir. Son olarak noktanın mantis başına göre kaç basamak kaydığı bulunur ve bu değer excess-4 gösterimiyle üs alanına yazılır. 2 3/4 sayısı ikilikte 10.11'dir. Mantis 1011, noktanın mantis başından iki basamak sağa kayması gerektiği için üs +2 (110) olur. Böylece sayı 0|110|1011 olarak saklanır. 3/8 sayısı ise .011'dir. Mantis ilk 1'den başladığı için 1100 olur ve nokta bir basamak sola kayacağından üs −1 (011) olur. Sonuç 0|011|1100'dür. Sağlama yapılırken nokta mantisin başına konur ve üs değeri kadar kaydırılır. Sola kaydırmada boşalan basamağa 0 gelir.

Kayan nokta biçiminin sınırı mantis alanının genişliğidir. 2 5/8 sayısı ikilikte 10.101'dir ve beş anlamlı bit gerektirir. Mantis dört bit olduğundan son bit düşer ve saklanan değer 0|110|1010, yani 2 4/8 olur. Bu duruma kesme hatası (truncation error) ya da yuvarlama hatası (round-off error) denir. Bilgisayar hata mesajı vermez. Sayı sessizce daha düşük hassasiyetle işlenir. Aynı nedenle 1/3 gibi 2'nin kuvvetleriyle tam yazılamayan sayılar da sınırlı bitle tam ifade edilemez. Gerçek bilgisayarlarda bu sorun daha geniş biçimlerle azaltılır. Single precision (32 bit) biçimde 1 bit işaret, 8 bit üs ve 23 bit mantis vardır, yaklaşık 7 ondalık basamak hassasiyet sağlanır. Double precision (64 bit) biçimde hassasiyet yaklaşık 15 ondalık basamağa çıkar.

Dersin ikinci bölümünde bilgisayarda hangi tür verilerin saklandığı ele alınır. Önce veri ile bilgi ayrılır: veri anlamlandırıldığında bilgi olur. Bilgisayar sayıları, karakterleri, renkleri, CPU

komutlarını, sesleri, resimleri, videoları, dosyaları ve programları 0 ve 1'lerle saklar. Her tür için belirli kurallar ve tablolar vardır.

Karakterler için en bilinen tablo ASCII'dir. Her karaktere bir sayı karşılık gelir. '0' karakteri 48, 'A' karakteri 65'tir ve diğerleri bunları sırayla izler. Hoca bu değerlerin ezberlenmemesini, bunun yerine karakterler arasındaki farkların kullanılmasını ister. Bir rakamın sayı olarak ve karakter olarak saklanması farklıdır: 3 sayısı 00000011 olarak, '3' karakteri ise 51 değeriyle 00110011 olarak saklanır. Benzer şekilde "333" metni üç karakter olduğu için 3 byte, 333 sayısı ise 1 byte'a sığmadığı için 2 byte kaplar. Bellekte alanlar 8 bitin katları olarak ayrıldığından 9 bit gibi bir alan kullanılamaz. Bu yüzden sayısal işlem yapılacak değerler daha verimli oldukları için sayı tiplerinde saklanır, metin içindeki sayılar ise karakter olarak tutulur. ASCII'nin 256 sembolü Türkçe karakterleri ve farklı alfabeleri karşılamadığı için UTF-8 kullanılır. UTF-8, ASCII'deki sembolleri aynı kodlarla ifade eder ve gerektiğinde daha geniş bit alanları kullanarak tüm dillerin karakterlerini kodlayabilir.

Resimler piksellerden oluşan, yüksekliği ve genişliği olan matrisler olarak saklanır. Her piksel 1 byte ile ifade edilirse 1920×1080 bir resim 1920 × 1080 byte yer kaplar. Saniyede 30 kare olan bir videoda bu miktar her saniye için 30 ile çarpılır. Ses ise genellikle zamana bağlı şiddet değerlerinden oluşan tek boyutlu bir dizi olarak saklanır. Bu veriler çok yer kapladığı için sıkıştırılır. Kayıpsız sıkıştırmada veri açıldığında birebir aynısı elde edilir. Kayıplı sıkıştırmada ise insanın algılayamayacağı ayrıntılar atılır. MP3 formatının duyulmayan sesleri saklamaması buna örnektir. Kayıplı yöntemler çoğunlukla ses, görüntü ve videoda kullanılır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.